

Documentação de Arquitetura e Requisitos

Projeto: Evolutto

Autor / Tech Lead: Allan Victor Rodrigues Souza

Stack Tecnológica: Angular (Frontend), Java / Spring Boot (Backend), PostgreSQL, Redis.

Padrão Arquitetural: MVC (Model-View-Controller) / Monólito Modular.

1. Visão Geral do Produto

O Evolutto é uma aplicação de gestão de hábitos gamificada. O diferencial competitivo do sistema é a expansão da gamificação tradicional para dinâmicas sociais (competições entre amigos) e um módulo exclusivo de controle parental, garantindo flexibilidade na validação e gestão de tarefas.

2. Arquitetura do Sistema

A aplicação adotará a arquitetura de **Monólito Modular** dividida em três fases de implantação, garantindo entregas de valor contínuas. O backend será estruturado estritamente utilizando o padrão **MVC (Model-View-Controller)**, assegurando uma separação clara entre a camada de apresentação (Controllers REST), regras de negócio (Services) e acesso a dados (Models/Repositories).

- Frontend (Views / UI):** Desenvolvido em Angular. Consumirá a API REST e gerenciará o estado local das telas, componentes de gamificação (barras de XP, loja) e dashboards parentais/sociais.
- Backend (Controllers & Models):** Spring Boot (Java). Focado em escalabilidade transacional e segurança.
- Persistência Relacional:** PostgreSQL para garantir consistência transacional (ACID) em saldos de moedas, XP e logs de auditoria parental.
- Cache e Leaderboards:** Redis para gerenciar sessões temporárias e o ranqueamento de alta concorrência dos grupos (Leaderboards), evitando sobrecarga de ordenação no banco relacional.

3. Regras de Negócio e Mecânicas

3.1. Sistema Core (Hábitos Bons e Lojinha)

- A dificuldade do hábito define o multiplicador de recompensa. Hábitos difíceis concedem mais XP e moedas do que hábitos fáceis.
- As recompensas da Lojinha são personalizadas e criadas pelo próprio usuário (ex: "30 min de videogame" = 50 moedas).
- A compra de um item debita as moedas do saldo do usuário.

3.2. Mecânica de Hábitos Ruins e Debuffs

- A execução de um hábito ruim não subtrai saldo de moedas nem de XP atual, protegendo a experiência do usuário contra frustrações extremas.
- **Penalidade (Debuff):** Ao registrar um hábito ruim, o sistema aplica um estado de *debuff* no usuário, reduzindo em **50% o ganho de XP nos próximos 10 hábitos concluídos**. Um contador na interface deixará claro quantos hábitos faltam para o debuff ser removido.

3.3. Sistema de Validação (Anti-Cheat)

O fluxo de mudança de estado da conclusão de um hábito varia conforme o contexto do usuário:

- **Modo Solo:** Baseado no sistema de honra. O usuário dá o *check* e o sistema processa os ganhos imediatamente.
- **Competições Sociais:** Continua baseado no sistema de honra, porém com um **Log de Auditoria Social**. Todas as conclusões são publicadas no feed do grupo fechado. A auditoria é feita pela pressão visual dos próprios amigos.
- **Controle Parental (Maker/Checker):** A criança (Maker) conclui a tarefa. O status vai para **PENDING_APPROVAL**. O XP e as moedas ficam retidos. O responsável (Checker) recebe a notificação, revisa e altera o status para **COMPLETED** (liberando a transação) ou **REJECTED**.

3.4. Castigos (Módulo Parental)

- **Congelamento de Loja:** Em vez de retirar moedas, os pais podem aplicar o status **FROZEN** na lojinha da criança.
- **Missões de Redenção:** Para remover o status **FROZEN**, os pais cadastram uma "Missão de Redenção" (ex: tarefa doméstica específica). Apenas após o fluxo de validação dessa missão, a loja é descongelada.

4. Escopo de Entrega (Faseamento)

- **Fase 1 - Core:** Autenticação, CRUD de hábitos bons e ruins, cálculo de XP/Moedas, transações na Lojinha, aplicação e controle do Debuff de -50%.
- **Fase 2 - Social:** Sistema de amizades, criação de Grupos, Leaderboards processados no Redis, log de auditoria visual de amigos.
- **Fase 3 - Parental:** Fluxo `PENDING_APPROVAL`, contas supervisionadas, congelamento de loja e missões de redenção.

5. Modelagem de Dados Principal (PostgreSQL)

Para dar suporte ao MVC, as seguintes Models/Entidades centrais serão mapeadas:

- **User** : Gerencia dados de login, `current_xp`, `current_coins`, `level`, e o atributo `debuff_counter`.
- **Habit** : `title`, `difficulty_level`, `type` (GOOD, BAD), `user_id`.
- **HabitLog** : Registro imutável. Contém `status` (COMPLETED, PENDING_APPROVAL, REJECTED).
- **Reward** : Item da lojinha com `title`, `cost`, `user_id`.
- **Group** & **GroupMember** : Estrutura para competições e amizades.
- **ParentalRelationship** : Mapeia `parent_id` e `child_id`.

6. Estrutura de Endpoints REST (Controllers)

- **HabitController**: `POST /api/v1/habits`, `POST /api/v1/habits/{id}/execute`
- **ShopController**: `GET /api/v1/shop`, `POST /api/v1/shop/buy/{rewardId}`
- **GroupController**: `GET /api/v1/groups/{id}/leaderboard` (Redis)
- **ParentalController**: `GET /api/v1/parental/pending-approvals`, `POST /api/v1/parental/approve/{logId}`, `POST /api/v1/parental/freeze-shop/{childId}`